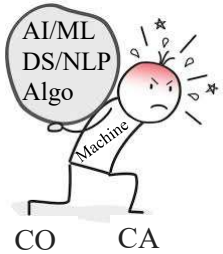
CS-2710-1 Spring 2026

Computer Organization and Systems (COS)

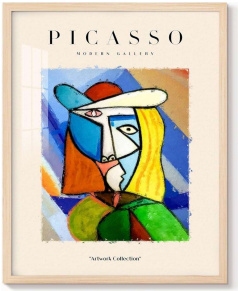


Lecture #11

11 March 2026

Ashoka University

BBB



"Computers are useless.
They can only give you answers."

Pablo Picasso (1881 – 1973)

Computer Arithmetic: Review

Review: IEEE 754 floating-point standard

1	8	23
S	E	M

- Leading "1" bit of significand is implicit
- Exponent is "biased" to make comparison easier
  - all 0s is smallest exponent; all 1s is largest
  - bias of 127 for single precision and 1023 for double precision
  - summary: $(-1)^{sign} \times (1+\text{significand}) \times 2^{exponent - bias}$
- Example 1 (Encoding):
  For a given decimal number, construct its FP-representation
  - decimal: - 0.75
  - → binary: - 0.11 = - 1.1 x 2^{-1} (normalized)
  - → Floating-point exponent = - 1 +127 = 126 = 01111110

IEEE single precision:

- 0.75 = `1 01111110 10000000000000000000000`

Decoding: Example

1	0111 1110	11000000000000000000000000000

- Sign bit is one:
  Number is negative
- Biased exponent E = 0111 1110 $|_2$ = 126;
- Actual exponent $e = E - 127 = -1 \rightarrow 2^{-1}$
- Significand → 1.11 (restored the hidden bit)
- The number = $-1.11 \times 2^{-1}|_2 = -0.111|_2 = -0.875|_{10}$

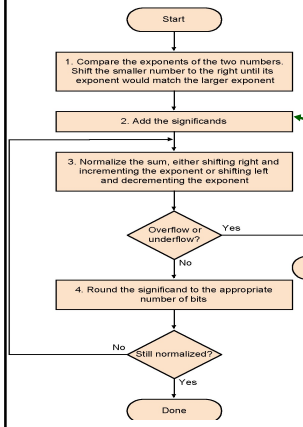
Summary of IEEE 754 Encoding

Single-Precision	Exponent = 8	Fraction = 23	Value
Normalized Number	1 to 254	Anything	$\pm (1.F)_2 \times 2^{E-127}$
Denormalized Number	0	nonzero	$\pm (0.F)_2 \times 2^{-126}$
Zero	0	0	± 0
Infinity	255	0	± ∞
NaN	255	nonzero	NaN

Double-Precision	Exponent = 11	Fraction = 52	Value
Normalized Number	1 to 2046	Anything	$\pm (1.F)_2 \times 2^{E-1023}$
Denormalized Number	0	nonzero	$\pm (0.F)_2 \times 2^{-1022}$
Zero	0	0	± 0
Infinity	2047	0	± ∞
NaN	2047	nonzero	NaN

Overflow leads to ± ∞; underflow (gradual) → denormals; underflow (abrupt) → ± 0

Floating-Point Addition



If one number is +ve and the other is –ve, addition of significands should be done using 2's complement and the result is reconverted into sign-magnitude form

A + B =

0	1000 0001	0000 0000 0000 0000 0000 000

A + B = 10.0 × 2^1; A + B = 1.0 × 2^2

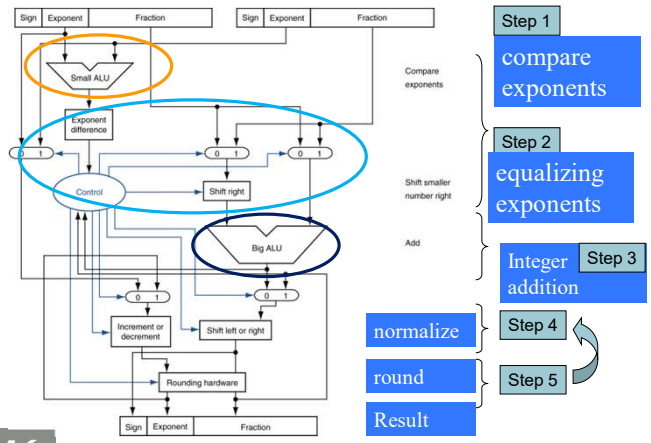
A: +0.1000 0000 0000 0000 0000 000 × 2^1

A: + 1.0000 0000 0000 0000 0000 000 × 2^0

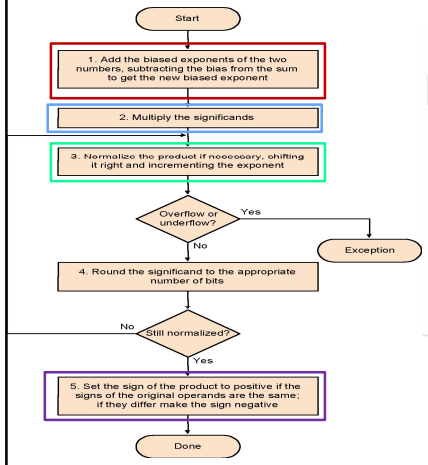
B: + 1.1000 0000 0000 0000 0000 000 × 2^1

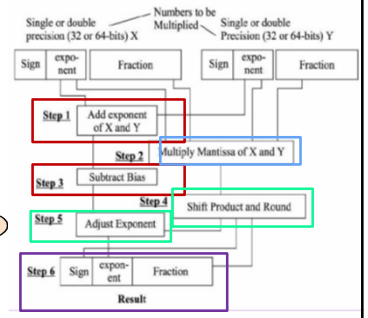
A	0	0111 1111	0000 0000 0000 0000 0000 000
B	0	1000 000	1000 0000 0000 0000 0000 000

FP Adder Hardware



Floating Point Multiplication



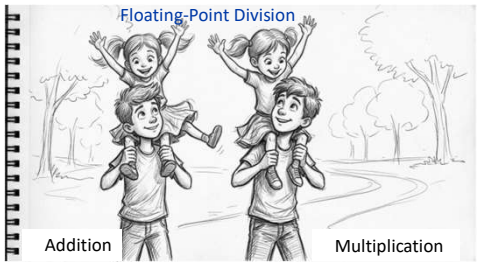


Agenda today …

- Floating-Point Addition
- Floating-Point Multiplication
- Floating-Point Division ?

Idea …

… Functional iteration methods transform FP-division into several FP-multiplications and additions



FP-Division (using multiplication and add/subtract)

- Divide A/B to produce C; Both A, B are in IEEE 754 float format
  - There is no notion of a remainder in FP division
- Algorithm
  - $E_C \leftarrow E_A - E_B + \text{bias}$
  - $S_C \leftarrow S_A / S_B$
  - S represents the significand with leading 1 restored
  - normalize, round, re-normalize

FP-Division via Goldschmidt Method

- Divide A/B to produce C; Both A, B are in float format
- Let us compute the reciprocal (1/B) of B
  - Then, we can use the standard FP-multiplication:

$$C = A/B = A * (1/B)$$

- Ignoring the exponent of B,
  - Let us compute $(1/S_B)$
- If B is a normalized floating-point number, $1 \leq S_B < 2$

Let $S_B = 1 + X$; $(0 \leq X < 1)$

$S_B = 1.11$;
$X = 0.11$

Example B:

1 | 0111 1110 | 11000000000000000000000000000

FP-Division via Goldschmidt Method

- Let us compute the reciprocal (1/B) of B
- Let us compute $(1/S_B)$ –> reciprocal of the significand
- If B is a normalized floating-point number, $1 \leq S_B < 2$

Let $S_B = 1 + X$; $(0 \leq X < 1)$

$$\frac{1}{S_B} = \frac{1}{1+X} = \frac{1}{1+1-X'}; \ (X' = 1 - X, X' \leqslant 1)$$

$$= \frac{1}{2-X'} = \frac{1}{2} * \frac{1}{1-\frac{X'}{2}}$$

$$= \frac{1}{2} * \frac{1}{1-Y}; \ \left(Y = \frac{X'}{2}, Y \leqslant \frac{1}{2}\right)$$

FP-Division via Goldschmidt Method

$$\frac{1}{S_B} = \frac{1}{1+X} = \frac{1}{1+1-X}; \ (X' = 1 - X; \ X' \leq 1)$$

$$= \frac{1}{2-X'} = \frac{1}{2} * \frac{1}{1-\frac{X'}{2}}$$

$$= \frac{1}{2} * \frac{1}{1-Y}; \ \left(Y = \frac{X'}{2}, Y \leqslant \frac{1}{2}\right)$$

Now, $\frac{1}{1-Y} = \frac{1+Y}{1-Y^2} = \frac{(1+Y)(1+Y^2)}{1-Y^4}$

$$= \ldots$$

$$= \frac{(1+Y)(1+Y^2) \ldots (1+Y^{16})}{1-Y^{32}}$$

$$\approx (1+Y)(1+Y^2)(1+Y^4) \ldots (1+Y^{16})$$

Y^{32} is negligible

FP-Division via Goldschmidt Method

$$\frac{1}{S_B} = \frac{1}{1+X} = \frac{1}{1+1-X'}; \ (X' = 1 - X; \ X' \leq 1)$$

$$= \frac{1}{2} * \frac{1}{1-Y}; \ \left(Y = \frac{X'}{2}, Y \leqslant \frac{1}{2}\right)$$

$$\approx \frac{1}{2} * (1+Y)(1+Y^2)(1+Y^4) \ldots (1+Y^{16})$$

B: 1 | 0111 1110 | 11000000000000000000000000000

- Use FP-multiplier to obtain Y^2, Y^4, Y^8, and Y^{16}
- Need 4 multiplications, and 5 additions;
- Need 4 more multiplications to generate (1/(1-Y));
- Compute $1/S_B$ by a right-shift through one-bit;
- Multiply it by S_A and compute exponent of C;
- Round, normalize, re-normalize to obtain C = A/B;

$S_B = 1.11$;
$X = 0.11$;
$X' = (1 - X) =$ 0.01 (in binary);
$Y = \frac{X'}{2} = 0.001$

FP-Division via Newton-Raphson Method

Divide A/B to produce C; Both A, B are in float format

Let us compute the reciprocal of B (= 1/B)

- Then, we can use the standard FP-multiplication: C = A/B = A * (1/B)

Ignoring the exponent of B, Let us compute ($1/S_B$)

If B is a normalized floating-point number, $1 \leq S_B < 2$; let $b = S_B$;

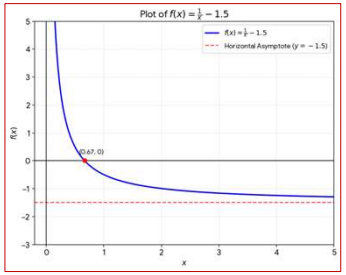
- Aim is to compute $1/b$

Let us create a function $f(x) = (1/x) - b$

- $f(x) = 0$, when $x = 1/b$

Problem of computing the reciprocal

- same as computing the root of f(x)

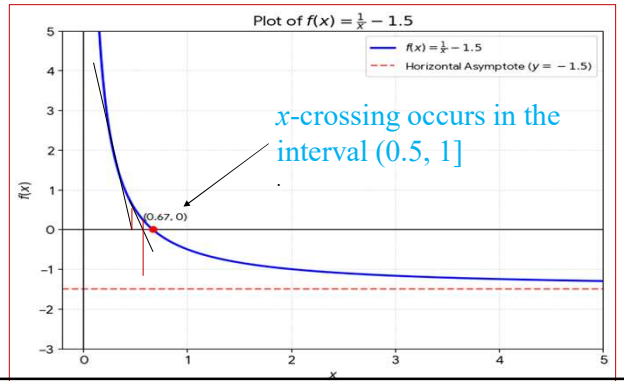


FP-Division via Newton-Raphson Method

Aim is to compute $1/b$; $f(x) = (1/x) - b$

- $f(x) = 0$, when $x = 1/b$ $\quad 1 \leq b = S_B < 2$;

- computing the reciprocal: same as computing the root of f(x)



Iterative method for computing ($1/b$)

- Start with an approximate value of x (0.5, 1] = x_0 (from a look-up table
  - Locate x_0 on the graph of f(x)
  - Draw a tangent to f(x) at (x_0, $f(x_0)$)
  - Let the tangent intersect the x axis at x_1
  - Draw another tangent at (x_2, $f(x_2)$)
- Keep repeating
  - Ultimately, we will converge to the root

FP-Division via Newton-Raphson Method

Aim is to compute $1/b$; $f(x) = (1/x) - b$; $\quad 1 \leq b = S_B < 2$;

- $f(x) = 0$, when $x = 1/b$

- computing the reciprocal: same as computing the root of f(x)

Newton's iteration for zero finding:
If x_i is an estimate for a zero of f, then x_{i+1} is a better estimate. To compute x_{i+1}, find the intersection of the x-axis with the tangent line to f at $f(x_i)$.

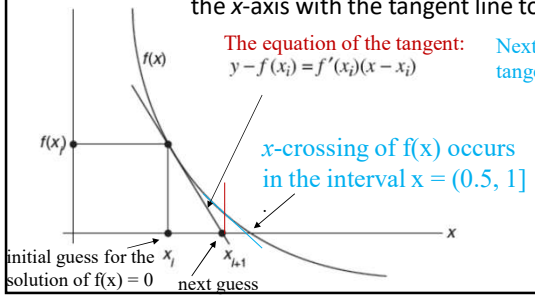
The equation of the tangent:
$y - f(x_i) = f'(x_i)(x - x_i)$

Next guess x_{i+1} : where the tangent crosses x-axis:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

$$= x_i - \frac{1/x_i - b}{-1/x_i^2}$$

$$= x_i(2 - x_i b)$$



FP-Division via Newton-Raphson Method

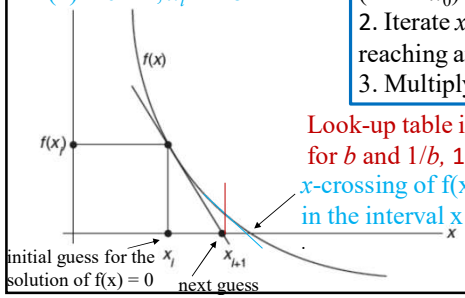
Aim is to compute $1/b$; $f(x) = (1/x) - b$; $1 \le b = S_B < 2$;

- $f(x) = 0$, when $x = 1/b$

- computing the reciprocal: same as computing the root of f(x)

$x_{i+1} := x_i(2 - x_i b)$

x_i is a guess solution for $f(x) = 0$ i.e., $x_i = 1/b$



Algorithm: C = A/B
1. Obtain an approximate value of $1/b$ (call it x_0) using a table lookup;
2. Iterate $x_{i+1} = x_i(2 - x_i b)$ until reaching an x_n that is accurate enough;
3. Multiply x_n by S_A to obtain S_C;

Look-up table in **binary** for b and $1/b$, $1 \le b < 2$;
x-crossing of f(x) occurs in the interval $x = (0.5, 1]$

b	$1/b$
1.00	1.00
1.01	0.11
1.10	0.10
1.11	0.10

Number of Iterations ..

Thus, division can be accomplished with only shift, multiply, and subtract operations

Aim is to compute $1/b$; $f(x) = (1/x) - b$;

- $f(x) = 0$, when $x = 1/b$; $1 \le b = S_B < 2$;

- computing the reciprocal: same as computing the root of f(x)

$x_{i+1} := x_i(2 - x_i b)$ $\qquad 1/2 \le x_0 \le 1$;

$$\text{error in } x_0 = \varepsilon(x_0) = x_0 - \frac{1}{b} = \left[\frac{bx_0 - 1}{b}\right] = x_0 - 1/b \le 1/2$$

$$\begin{aligned}\text{error in } x_1 = \varepsilon(x_1) &= (bx_1 - 1)/b \\ &= [b(2x_0 - bx_0^2) - 1]/b \\ &= -(bx_0 - 1)^2/b \\ &= -\varepsilon(x_0)^2 b; \\ |\varepsilon(x_1)| &\sim |\varepsilon(x_0)|^2\end{aligned}$$

- $|\varepsilon(x_5)| \sim 1/2^{32}$;
- Hence, 5 iterations will be enough for single-precision FP-arithmetic as we have 23-bit mantissa;

The error thus reduces by a power of 2 in every iteration!

Arithmetic for Computers: Review

- Operations on integers
  - Addition and subtraction
  - Multiplication and division
  - Dealing with overflow
  - Hardware implementation of ALU
- Floating-point real numbers
  - Representation and operations
  - Overflow and underflow
  - Hardware implementation of FP-operations

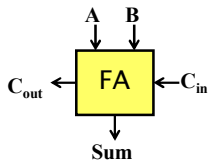
Why integer adders are so important?

- Addition: a fundamental operation
  - Basic block of most arithmetic operations
  - Address calculation
  - FP adders and multipliers all need integer addition
- Faster and faster
- How?
  - Architectural-level optimization
  - Gate-level optimization
  - Speed/area trade-off

How to speed up computer arithmetic?

Review: 1-bit adder

- One-bit Full Adder:



$$\text{Sum} = A \oplus B \oplus C_{in}$$

$$C_{out} = A.B + B.C_{in} + A.C_{in}$$

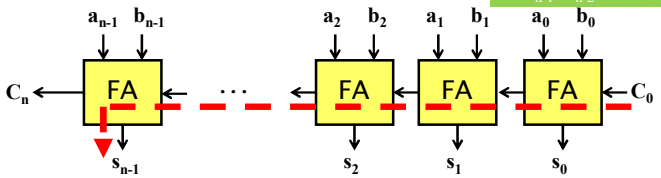
C_{in}	A	B	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

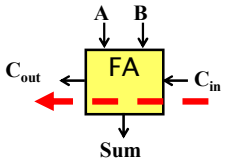
n-bit Ripple-Carry Adder (RCA)

- To add two *n*-bit numbers

A: $a_{n-1}, a_{n-2}, \ldots\ldots, a_1, a_0$

B: $b_{n-1}, b_{n-2}, \ldots\ldots, b_1, b_0$





- N-bit RCA delay = $n * \delta$
- δ = (delay of 1-bit FA)

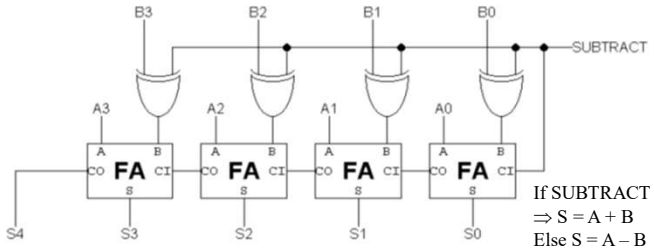
Critical path in an *n*-bit ripple-carry adder
→ CCT needs to accommodate long delay
Cost = $O(n)$; delay = $O(n)$

Subtraction using 2's complement
(Compute A – B)

Using 2's complement representation: -B = ~B + 1

~ = bit-wise complement

So let's build an arithmetic unit that does both addition and subtraction. Operation selected by *control input*:

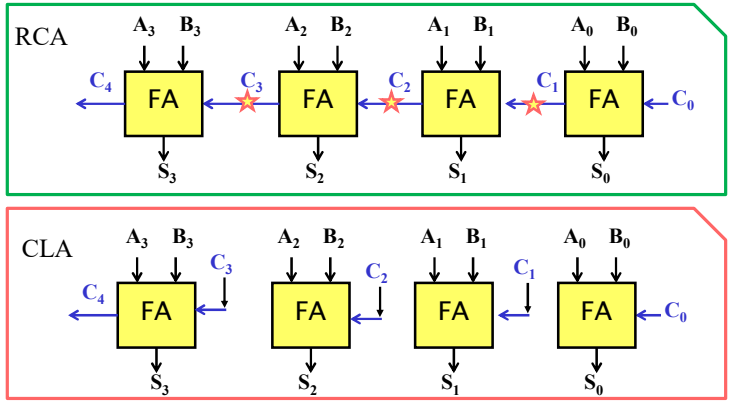


If SUBTRACT = 0
⇒ S = A + B
Else S = A – B

Possible solutions to mitigate carry-propagation delay

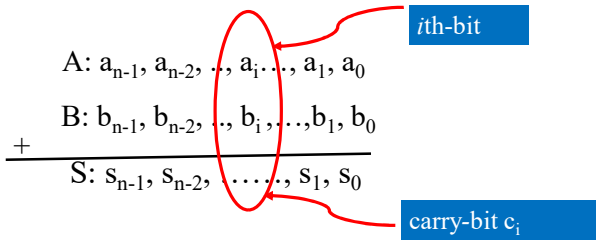
1. Speed-up propagation using techniques such as
   - carry lookahead
   - carry-select
   - logarithmic adder
2. Limit carry propagation to within a small number of bits
3. Trade-off: speed (or inversely, delay) *versus* logic cost

Idea: Carry-Lookahead Adder (CLA)



Instead of RCA, use *n independent* 1-bit FA-modules working in parallel; no signal propagation through stages; directly generate all carry-bits from inputs

Carry Lookahead Adder (CLA): Basic Idea



At the *i*th *stage*, carry will be generated for propagation to the next stage if and only if
$a_i = b_i = 1$; => $g_i = a_i\, b_i$ ==> carry-generate function

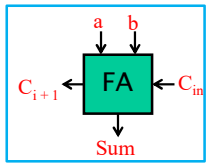
or either a_i or $b_i = 1$, and previous carry c_{i-1} arrives here;
=> $p_i = a_i + b_i$ => carry-propagate function

Carry-Lookahead Adder

We can visualize carry "generate" and "propagate" functions from another perspective:

$$C_{i+1} = a.b + b.\, c_{in} + a.\, c_{in}$$

$$= a.b + c_{in}.\, (a+b)$$



- A column will generate a carry out if a_i AND b_i are both 1, i,e., $g_i = a_i\, b_i$
- A column will propagate a carry if A_i or B_i is 1, i.e., $p_i = a_i + b_i$

- The carry out of a column (C_i) is:

$$c_{i+1} = a_i b_i + (a_i + b_i)c_i = g_i + p_i\, c_i$$

Basic Signals

Generate signal: $g_i = a_i b_i$

Propagate signal: $p_i = a_i + b_i$

Carry recurrence

$$c_{i+1} = g_i + c_i p_i$$

4-bit Carry-Lookahead Adder

$$c_4 = g_3 + g_2 p_3 + g_1 p_2 p_3 + g_0 p_1 p_2 p_3 + c_0 p_0 p_1 p_2 p_3$$

$$c_3 = g_2 + g_1 p_2 + g_0 p_1 p_2 + c_0 p_0 p_1 p_2$$

$$c_2 = g_1 + c_1 p_1$$

$$c_1 = g_0 + c_0 p_0$$

$g_i = a_i b_i$
$p_i = a_i + b_i$

$c_{i+1} = g_i + c_i p_i$

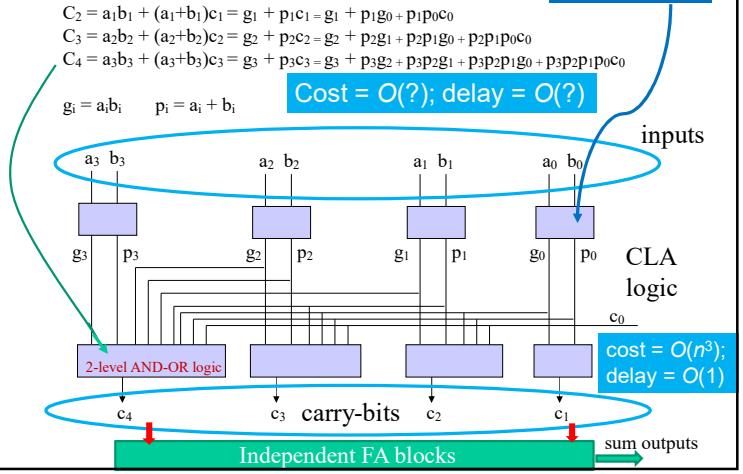
Carry Lookahead Adder: Analysis

one AND-gate
and
one OR-gate

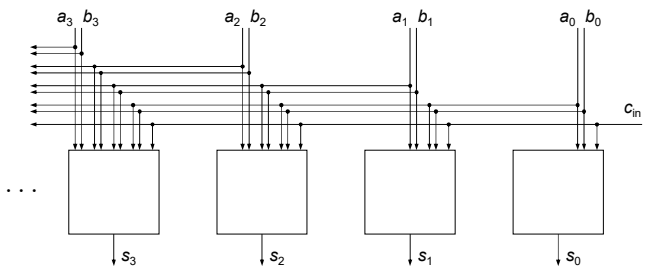
$C_1 = a_0 b_0 + (a_0+b_0)c_0 = g_0 + p_0 c_0$
$C_2 = a_1 b_1 + (a_1+b_1)c_1 = g_1 + p_1 c_1 = g_1 + p_1 g_0 + p_1 p_0 c_0$
$C_3 = a_2 b_2 + (a_2+b_2)c_2 = g_2 + p_2 c_2 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0$
$C_4 = a_3 b_3 + (a_3+b_3)c_3 = g_3 + p_3 c_3 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0$

$g_i = a_i b_i$ $p_i = a_i + b_i$

Cost = $O(?)$; delay = $O(?)$



Full Carry Lookahead



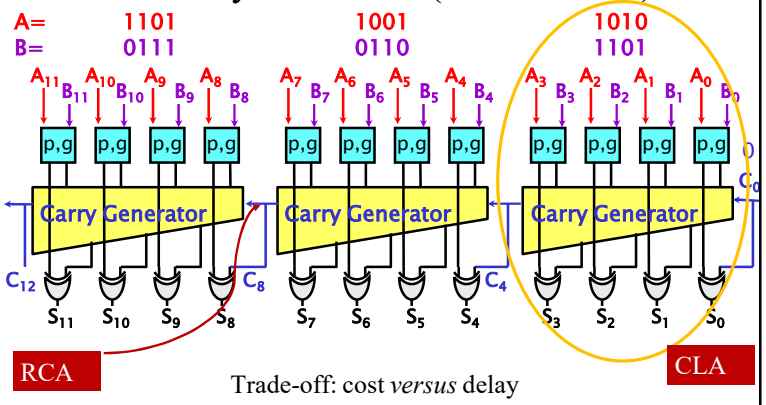
Theoretically, it is possible to derive each sum digit directly from the inputs that affect it

Full carry-lookahead adder is impractical for large *n*

Solution 1: *n*-bit Hybrid CLA Adder

- Implementation of lookahead for the complete adder is impractical because of cost
  - Divide *n* stages into smaller groups
  - Full carry lookahead within each group
  - Ripple carry among groups
  - Compromise between cost and delay

12-Bit Hybrid Adder (CLA + RCA)



Trade-off: cost *versus* delay

Another solution: 12-Bit Hybrid (RCA + CLA)??
Cost = *O*(?); delay = *O*(?)